

Patata

- **Limite di tempo:** 10 minuti
- **Ambiente:** una GPU (≈16 GB VRAM), senza internet
- **Dimensione della soluzione:** `solution.ipynb` ≤ 1 MB
- **Spazio di archiviazione:** 5 GB

Compito

Il tuo amico propone di giocare a un gioco di indovinelli. Lui, in qualità di giudice, sceglie una parola nascosta da un vocabolario fisso, e tu devi trovarla in al massimo 30 turni. A ogni turno il giudice confronta due parole e comunica quale sia semanticamente più vicina alla parola nascosta. Ogni partita inizia dalla coppia fissa `lamp` vs `potato`, perché sono due delle cose preferite del tuo amico. Il tuo programma quindi propone una nuova parola. La parola vincitrice del confronto viene mantenuta e confrontata con la tua proposta successiva. Vinci una partita nel momento esatto in cui proponi la parola nascosta. Il confronto non distingue tra maiuscole e minuscole. Ogni parola che proponi deve appartenere a `dataset/vocabulary.json`.

È disponibile un esempio completo in `solution.ipynb` con il protocollo e il caricamento dei dati. Puoi modificare la classe `PublicEmbeddingPlayer`. Il tuo programma viene inizializzato una sola volta e gioca ogni partita in un'unica esecuzione; il protocollo crea un nuovo `PublicEmbeddingPlayer` all'inizio di ogni partita.

Il giudice

Il tuo programma invia un oggetto JSON al Giudice e il Giudice risponde con un oggetto JSON.

Un esempio svolto, in cui la parola nascosta è mostrata esclusivamente per spiegare il protocollo:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}                                     <- matches: game won
-> {"status": "win"}
```

I turni sono indicizzati da 1 a 30.

Le opzioni di `verdict` sono `first`, che significa che `word1` è più vicina, `second`, che significa che `word2` è più vicina, oppure `same`, che significa che entrambe le parole sono ugualmente vicine alla parola nascosta.

`winner_word` è la parola mantenuta per il confronto successivo. In caso di verdetto `same`, rimane la prima parola.

Dataset

Condivisi da ogni split:

- `dataset/vocabulary.json` — 1602 parole minuscole univoche. La parola nascosta è sempre una di queste.
- `dataset/public_embeddings.npy` — `float32`, di forma `(1602, 2560)`. La riga `i` corrisponde alla parola `i` nel vocabolario. Questi sono *embedding pubblici*; il giudice utilizza una rappresentazione privata diversa.

Gli split sono insiemi di parole nascoste:

Split	Parole	Risposte	Utilizzalo per
<code>test_public</code>	120	✔ <code>dataset/test_public.json</code>	eseguire la tua soluzione e calcolare autonomamente il punteggio
<code>test_leaderboard_a</code>	120	nascoste	leaderboard in tempo reale
<code>test_leaderboard_b</code>	120	nascoste	classifica finale

Non esiste uno `split train` — non viene effettuato alcun fitting su righe etichettate.

Modelli forniti

Con il compito vengono forniti due modelli di embedding preaddestrati, che possono essere utilizzati:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Entrambi devono essere caricati dal rispettivo percorso locale; un ID dell'hub di Hugging Face come "BAAI/bge-m3" avvia un download e non funziona, perché la valutazione avviene offline. Ogni directory contiene un file `example.py` eseguibile che mostra la chiamata offline.

Librerie disponibili: `numpy`, `torch`, `sentence-transformers`. Nessun accesso a internet, nessun download, nessun altro pacchetto.

Output

Nessuno. Questo è un compito interattivo: la tua soluzione non scrive alcun file di risposta; comunica con il giudice tramite `stdin/stdout` come descritto sopra.

Metrica

Una partita risolta al turno t ottiene $1.0 - 0.02 \times \max(0, t - 10)$; una partita non risolta entro 30 turni ottiene 0. Pertanto, i turni 1–10 ottengono 1.00, il turno 20 ottiene 0.80, il turno 30 ottiene 0.60.

Il punteggio del compito è la media dei punteggi delle partite $\times 100$, compresa tra 0.00 e 100.00.

Il limite di 10 minuti costituisce un unico budget che comprende l'avvio, la preparazione e tutte le 120 partite nel set di test.

Come inviare

1. Apri `solution.ipynb`, modifica `PublicEmbeddingPlayer` ed esegui tutte le celle per assicurarti che funzioni.
2. Facoltativamente, verificalo localmente: `python local_test.py solution.ipynb --limit 5`. Il giudice locale utilizza gli embedding *pubblici*, quindi il suo punteggio è solo indicativo.
3. Salva `solution.ipynb`.
4. Apri la scheda Git nella barra laterale sinistra di JupyterLab.
5. Aggiungi `solution.ipynb` all'area di staging (l'icona + accanto al file).
6. Inserisci un messaggio di commit e fai clic su Commit.
7. Fai clic sull'icona della nuvola con la freccia verso l'alto per eseguire il push.
8. Torna a questa pagina Contest e fai clic su Submit, assicurandoti che il messaggio di commit corrisponda a quello che hai fornito.

Invia esattamente un file, denominato `solution.ipynb`, che includa ogni preparazione e inferenza necessaria.